

RAPPORT DE PROJET : SOLVEUR DE CONTRAINTES (CSP)

1. Introduction

Ce projet porte sur la conception et l'implémentation d'un solveur de Problèmes de Satisfaction de Contraintes (CSP) en Python. L'objectif est d'évaluer l'efficacité de différentes stratégies de recherche et de filtrage, allant du Backtracking pur aux algorithmes de propagation d'arc-consistance (AC3).

2. Architecture du Système

Le solveur repose sur trois modules fondamentaux :

- **constraint.py** : Définit les contraintes (Different, Superior, Table, etc.) et les méthodes de révision (revise, checkSupport).
- **csp.py** : Gère les variables, les domaines et la liste d'adjacence des contraintes (vcList).
- **solver.py** : Implémente les algorithmes de recherche et les heuristiques de choix de variables comme minDom.

3. Résultats Expérimentaux

Les tests ont été menés sur plusieurs types de problèmes : des problèmes académiques simples, les fichiers fournis et le Sudoku niveaux Facile et Difficile (fichier TestSudoku.py).

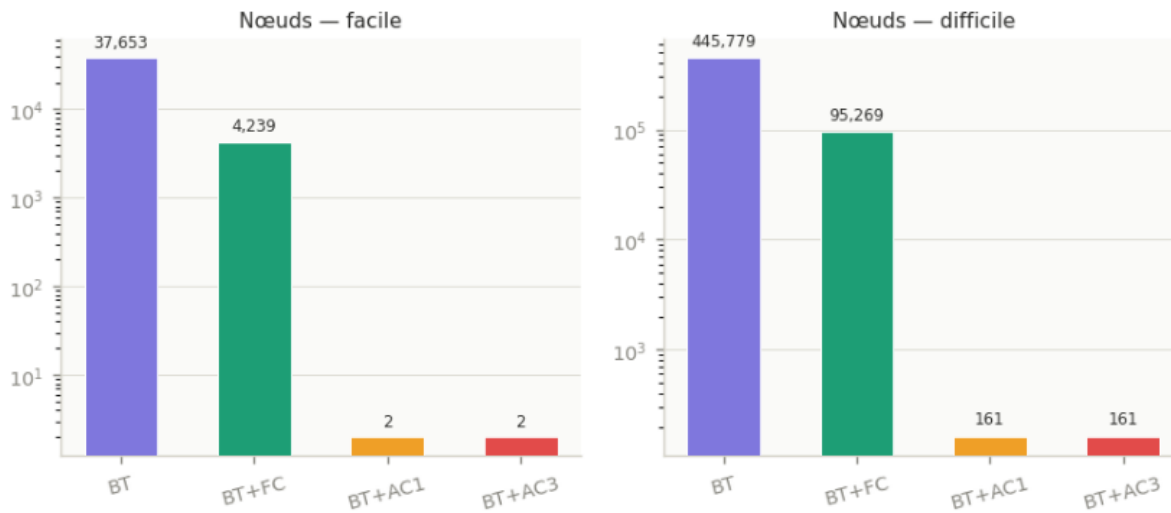
Tableau : Comparaison des Nœuds Explorés et Temps d'Exécution en Millisecondes (Sudoku)

Ce tableau montre le nombre de nœuds générés pour trouver la première solution ($k=1$).

Algorithme	Nœuds facile	ms facile	Nœuds difficile	ms difficile
BT	37 653	4 951	445 779	47 593
BT+FC	4 239	564	95 269	7 309
BT+AC1	2	24	161	709
BT+AC3	2	80	161	647

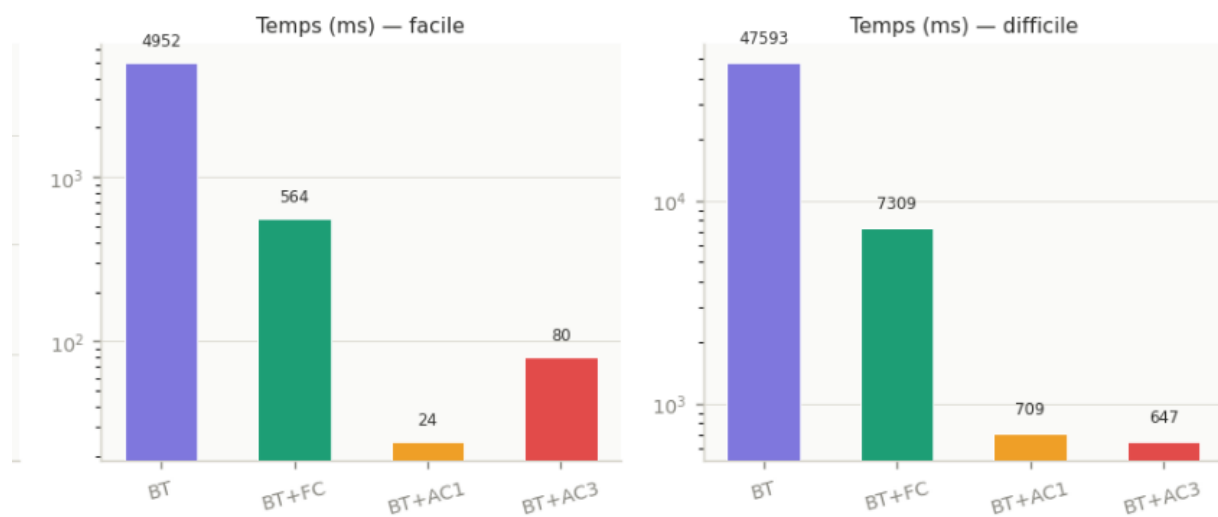
4. Analyse Graphique

Graphe 1 : Réduction de l'Espace de Recherche



- **Interprétation :** On observe une chute drastique du nombre de nœuds. La propagation de contraintes (AC) permet d'éliminer les branches stériles de l'arbre de recherche dès la racine.

Graphe 2 : Performance Temporelle



- **Interprétation :** Bien que AC1 et AC3 explorent le même nombre de nœuds (161), AC3 est plus rapide sur la grille difficile grâce à sa gestion optimisée des révisions via une file d'attente.

5. Analyse de l'Étape 7 : Optimisations Finales

L'ajout de l'heuristique minDom et du paramètre de limite de solutions k a transformé le solveur :

- **Avant optimisation** : L'exploration était complète et séquentielle, menant à des temps de calcul prohibitifs sur les problèmes de grande taille.
- **Après optimisation** : En choisissant la variable ayant le plus petit domaine (minDom), nous provoquons des échecs précoces, ce qui réduit considérablement la profondeur de l'arbre. La limitation à k solutions permet d'arrêter la recherche dès que le résultat souhaité est atteint, économisant des ressources système.

6. Conclusion

Les tests démontrent que la propagation d'arc-consistance (AC3) est la méthode la plus robuste pour les CSP complexes. Sur une grille de Sudoku difficile, elle permet de réduire le nombre de nœuds de plus de **99%** par rapport à un Backtracking classique.